Mineração de Métricas DORA em Repositórios Populares do GitHub

Enzo Casaes[1], **Cauê**[1], **Leonardo de Freitas Viana**[1]

[1]Instituto de Ciências Exatas e Informática (ICEI)
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil

Abstract. *This paper measures the four DORA metrics (deployment frequency, lead time for changes, change failure rate and time to restore) from objective GitHub traces in 300 popular repositories over a 12-month window. Most repositories fall into the High or Medium categories. Deploying more often is not associated with failing more: the apparent correlation with the delivery-based failure rate is an artifact of the metric's 7-day window. Project age is the characteristic most associated with performance, and between 21% and 27.7% of repositories change category depending on the proxies adopted.*

Resumo. *Este trabalho mede as quatro métricas DORA (frequência de deploy, lead time, change failure rate e tempo de recuperação) a partir de rastros objetivos do GitHub em 300 repositórios populares, numa janela de 12 meses. A maioria dos repositórios é High ou Medium. Publicar com mais frequência não está associado a falhar mais: a correlação aparente com o CFR baseado em entregas é efeito da janela de 7 dias da própria métrica. A idade do projeto é a característica mais associada ao desempenho, e entre 21% e 27,7% dos repositórios mudam de categoria conforme os proxies adotados.*

1. Introdução

O grupo DORA (DevOps Research and Assessment) propôs quatro métricas — frequência de deploy, lead time for changes, change failure rate e tempo de recuperação — que se tornaram o padrão de mercado para medir o desempenho de entrega de software [**?**]. Os relatórios Accelerate State of DevOps [**?**] classificam times em Elite, High, Medium e Low a partir delas, mas os dados vêm de questionários respondidos pelos próprios times. Neste laboratório o grupo mede as mesmas quatro métricas a partir de rastros objetivos deixados no GitHub (releases, commits e execuções do GitHub Actions) em 300 repositórios populares, ao longo de uma janela fixa de 12 meses (01/10/2025 a 30/09/2026).

A questão de fundo é dupla. Primeiro, saber como projetos open source populares se distribuem na escala DORA quando medidos por dados de repositório, e não por autodeclaração. Segundo, verificar se as métricas sobrevivem à operacionalização: "deploy", "falha" e "recuperação" não existem como eventos no GitHub e precisam ser aproximados por proxies — e proxies diferentes podem contar histórias diferentes.

1.1. Questões de Pesquisa e hipóteses informais

Tabela 1. Questões de pesquisa e hipóteses.

RQ	Pergunta	Hipótese informal (antes dos dados)
RQ 01	Qual a frequência de deploy (releases publicadas por semana) dos repositórios?	H1. A maioria fica entre Medium e High; Elite (≥ 7 releases/semana) é raro em projetos que versionam por release.
RQ 02	Qual o lead time for changes, nas variantes (a) por release e (b) por commit?	H2. Mediana entre 1 semana e 1 mês (Medium); a variante (b) é menor que a (a), porque um único commit antigo domina a (a).
RQ 03	Qual a change failure rate, nas variantes (a) proxy de CI e (b) proxy de entrega?	H3. As duas variantes divergem, pois medem coisas diferentes (pipeline quebrado $\times$ release seguida de correção).
RQ 04	Qual o tempo de recuperação de uma falha no CI do branch padrão?	H4. Horas, e não dias (High): um build quebrado no branch padrão bloqueia o time e é corrigido rápido.
RQ 05	Repositórios com maior frequência de deploy têm maior ou menor taxa de falha?	H5. Não há trade-off relevante: correlação com o CFR (a) no máximo fraca ($\lvert\rho\rvert < 0{,}3$). Uma correlação forte com o CFR (b) seria efeito da definição da métrica (quem publica mais tem mais releases "seguidas em até 7 dias"), e não de mais falhas.
RQ 06	Quais características do repositório estão associadas a melhor desempenho DORA?	H6. Mais contribuidores $\rightarrow$ mais deploys, lead time menor e CFR (a) maior; projetos mais antigos $\rightarrow$ menos deploys; linguagens com publicação automatizada de pacotes (TypeScript, JavaScript, Go, Rust) $\rightarrow$ mais deploys que C/C++.
RQ 07	A classificação DORA muda conforme a combinação de proxies (C1, C2, ...)?	H7. A categoria geral muda para parte relevante da amostra ($\geq 20\%$ dos repositórios em alguma combinação). A métrica mais sensível é o CFR: trocar o (a) pelo (b) rebaixa muitos repositórios, pois o CI quebra pouco mas releases corretivas são frequentes. Incluir pré-releases (C2) eleva a frequência de deploy e sobe alguns repositórios, sem mudar o quadro geral (maioria High/Medium).
RQ 08 (bônus)	Que proporção das releases é corretiva (rework rate) e como ela se relaciona com as outras métricas DORA?	Exploratória: sem hipótese registrada antes dos dados.

Contribuições além do enunciado (30%), detalhadas na Seção 3.5: (i) decomposição do CFR (b) em oportunidade $\times$ condicional, que testa se a correlação da RQ 05 é real ou artefato da métrica; (ii) controle de confusão entre idade e contribuidores por correlação parcial de Spearman na RQ 06; (iii) inferência com correção de Holm, intervalos de confiança e tamanho de efeito em todos os testes; (iv) quatro versões concorrentes da heurística de release corretiva, avaliadas por F1; (v) RQ 08 bônus, rework rate; (vi) pipeline reprodutível com cache, retomada, 423 testes automatizados e conferência cruzada em Python.

2. Contexto

Este é o Lab03 da disciplina Laboratório de Experimentação de Software (Engenharia de Software, 6º período), desenvolvido em três sprints (Lab03S01 a Lab03S03) e acompanhado pelo board do GitHub Projects mantido desde o Lab01. Assim como no Lab01, o objeto de estudo são repositórios populares do GitHub, mas agora a pergunta é sobre processo de entrega e não sobre características estáticas. Os dados produzidos aqui (o dataset de 300 repositórios e a classificação DORA) servem de base para os laboratórios seguintes.

A base conceitual é o livro Accelerate [**?**] e os relatórios State of DevOps [**?**], de onde vêm as quatro métricas e as faixas de classificação; a medição segue os princípios gerais de [**?**]. A tabela de cortes usada foi a fixada pela disciplina (Tabela 2), idêntica para toda a turma, para que os resultados de grupos diferentes sejam comparáveis. A categoria geral do repositório é a mediana dos pontos das quatro métricas (Elite = 4, High = 3, Medium = 2, Low = 1), arredondada para baixo.

Tabela 2. Cortes de classificação DORA fixados pela disciplina (adaptados do State of DevOps).

Métrica	Elite	High	Medium	Low
Deployment frequency	≥ 7 / semana	≥ 1 e < 7 / semana	≥ 1 / mês e < 1 / semana	< 1 / mês
Lead time (mediana)	< 1 dia	1 a < 7 dias	7 a < 30 dias	≥ 30 dias
Change failure rate	$\leq 15\%$	$> 15\%$ e $\leq 30\%$	$> 30\%$ e $\leq 45\%$	$> 45\%$
Tempo de recuperação (mediana)	< 1 hora	1 hora a < 1 dia	1 dia a < 1 semana	≥ 1 semana

Um ponto importante do contexto é que open source não faz "deploy" no sentido de produção. Aqui uma release publicada (não-draft, não-prerelease) é tratada como deploy, e uma execução de CI que falha no branch padrão é tratada como falha. Essas aproximações são justamente o que a RQ 07 (análise de sensibilidade) e a validação manual da seção 6 do enunciado colocam à prova.

3. Metodologia

3.1. Principais Desafios

- **Limite de 1.000 resultados da busca do GitHub.** A busca `stars:>1000` devolve no máximo 1.000 itens por consulta. A seleção foi fatiada por faixas de estrelas derivadas dos próprios dados (o teto de cada fatia é a menor contagem de estrelas da anterior, inclusive, com deduplicação por id), até reunir 1.500 candidatos de forma determinística.
- **Rate limit e volume de requisições.** O lead time exige uma comparação (`compare/{anterior}...{R}`) por release — mediana de 23 por repositório — e os workflow runs uma consulta por mês. O cliente respeita `X-RateLimit-Remaining/Reset` e `Retry-After`, repete falhas 5xx com backoff exponencial (1 s a 16 s) e tem timeout explícito: na primeira coleta real uma requisição ficou 17 minutos presa.

- **Teto de 1.000 runs por consulta filtrada.** Repositórios muito ativos passam desse teto; a coleta foi fatiada por mês e as fatias que ainda estouraram ficam registradas em `truncated_slices`.
- **Interrupções longas.** Coletas de várias horas foram interrompidas por cota, rede ou Ctrl+C. Sem cache, cada recomeço custaria a cota inteira de novo.
- **Definição operacional de "falha" e "recuperação".** Não há evento de falha em produção no GitHub. Foi preciso tratar censura à direita (falha nunca corrigida dentro da janela), censura à esquerda (falhas no início da janela sem sucesso anterior) e datas inconsistentes (commits com data posterior à release por rebase/squash; episódios com duração negativa).
- **Identificar release corretiva sem rótulo.** O CFR (b) depende de saber se a release seguinte "corrige um defeito recente", o que exige uma heurística e uma validação manual com três avaliadores independentes.

3.2. Tomadas de Decisão

Tabela 3. Principais decisões metodológicas.

Decisão	Escolha	Trade-off / justificativa
Critério de inclusão	≥ 5 releases e ≥ 50 workflow runs válidos na janela; GitHub Actions ativo	Critério da seção 3 do enunciado. Amostra final = os 300 elegíveis com mais estrelas.
Janela	01/10/2025 a 30/09/2026 (365 dias, 52,14 semanas), UTC	Mesma para todos os repositórios; idade medida no fim da janela, não no dia da coleta.
Deploy	Release com `draft = false` e `prerelease = false`	Pré-releases ficam para a variante C2 da RQ 07.
Lead time	(a) `published_at` − commit mais antigo da release; (b) por commit. Valor = mediana	(a) é usada na classificação (C1); (b) é menos sensível a um commit esquecido.
CFR (a)	falhas / (falhas + sucessos) dos runs `push` no branch padrão	Mede falha de pipeline, não de produção — ameaça de construto discutida na Seção 4.3.
CFR (b)	Release R falhou se a seguinte for corretiva e sair em ≤ 7 dias	Releases dos últimos 7 dias são censuradas; heurística padrão v3; a validação apontou a v1 como a de maior F1 (0,611), ainda abaixo de 0,70.
Recuperação	`updated_at` do sucesso − início da 1ª falha, por workflow; mediana	Episódios censurados ficam fora da mediana, mas sua proporção é reportada.
Métrica não medida	Fica fora da mediana da categoria geral	Repositório cujo CI nunca quebrou não vira Low (seria punição) nem Elite (não foi medido).
Estatística	Mediana e IQR; Spearman; Kruskal–Wallis e Mann–Whitney; Holm	Distribuições muito assimétricas — médias seriam puxadas por poucos repositórios extremos.
Ordem das etapas	runs antes de leadtime	O 2º critério do funil fecha a amostra antes da etapa cara: lead time é medido nos 300, e não nos $\sim$700 que só passaram pelo critério de releases.
Limite de WIP	3	Um item por integrante.

Configuração do processo. Board no GitHub Projects (v2) com as colunas **Backlog** → **To Do** → **Doing** → **Review** → **Done**. Política de WIP na coluna Doing: no máximo uma tarefa por integrante (limite de WIP = 3). Cada integrante commita só nos arquivos da sua Issue; na rotulagem, cada avaliador commitou apenas o próprio arquivo de rótulos, para preservar a independência exigida pelo kappa.

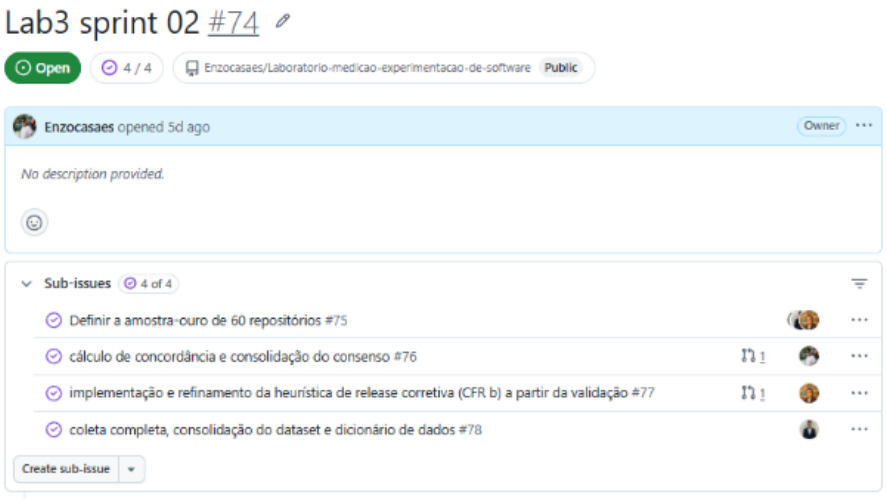


Figura 1. Print do processo: Issue #74 (Lab3 sprint 02) e suas sub-issues #75 a #78, todas concluídas.

3.3. Ferramentas

- **Node.js 22** (ES modules, `fetch` nativo, sem dependências externas) para todo o pipeline: `node src/pipeline.js -config config.json`, com etapas selecionáveis por `-etapa` e scripts `npm run analise:dora`, `analise:rq05`, `analise:rq06` e `validacao`.
- **API REST do GitHub** [?] acessada por cliente próprio (`src/github/cliente.js`), conforme a proibição de bibliotecas prontas como PyGithub. Endpoints: `search/repositories`, `actions/workflows`, `contributors`, `releases`, `compare` e `actions/runs`.
- **Cache em disco** (`data/cache`): um JSON por requisição, gravação atômica, sem o token — permite retomar a coleta e reproduzi-la offline.
- **node:test** — 423 testes automatizados, todos passando, incluindo uma API falsa (`test/apoio/api-falsa.js`) para testar paginação, rate limit e erros sem rede.
- **Python 3 com NumPy, SciPy e statsmodels** (`scripts/conferir_rq05_rq06.py` e `conferir_kappa.py`) para recalcular de forma independente Spearman, Kruskal–Wallis, Mann–Whitney, Holm e o kappa de Fleiss.
- **Matplotlib** para as figuras deste relatório; **CSV + Markdown** gerados pelo próprio pipeline (`docs/analise-*.md`) como fonte única dos números.
- **GitHub Projects (v2)** para o processo, no repositório `https://github.com/Enzocasaes/Laboratorio-medicao-experimentacao-de-software`.

3.4. Tabela de Métricas

Tabela 4. Métricas, definições operacionais e fontes.

RQ	Métrica	Definição operacional	Unidade	Fonte
RQ 01	Deployment frequency	releases (`draft = false`, `prerelease = false`, `published_at` na janela) ÷ 52,14 semanas	releases/ semana	API `releases`
RQ 02 (a)	Lead time por release	mediana, nas releases da janela, de `published_at(R)` − data do commit mais antigo de R (commits de `compare/anterior...R`)	dias	API `compare`
RQ 02 (b)	Lead time por commit	mediana de `published_at(R)` − `commit.author.date` de todos os commits de todas as releases	dias	API `compare`
RQ 03 (a)	CFR, proxy de CI	failure + timed_out + startup_failure ÷ (falhas + success), runs `push` no branch padrão; cancelados/skipped ignorados	proporção	API `actions/runs`
RQ 03 (b)	CFR, proxy de entrega	releases cuja seguinte é corretiva e sai em ≤ 7 dias ÷ releases avaliadas (sem censuradas e indeterminadas)	proporção	`releases` + heurística
RQ 04	Tempo de recuperação	mediana, por repositório, de `updated_at(sucesso)` − `run_started_at(1ª falha após um sucesso)`, dentro do mesmo workflow; + proporção de censurados	horas	API `actions/runs`
RQ 05	Associação frequência × falha	ρ de Spearman, IC 95% (Fisher, EP de Bonett-Wright [**?**]), Holm [**?**] na família de 6 pares	ρ	`dataset.csv`
RQ 06	Fatores × desempenho	Kruskal–Wallis [**?**] (ε^2) por fator × métrica; Mann–Whitney Q4 × Q1 e linguagem × C/C++ (δ de Cliff [**?**]); Holm por família	ε^2, δ	`dataset.csv`
RQ 06	Fatores	linguagem principal; contribuidores (inclui anônimos, via última página do Link); idade no fim da janela; estrelas — em quartis	—	API `search`, `contributors`
RQ 07	Sensibilidade da classificação	categoria DORA em cada combinação de proxies (C1 = release + LT (a) + CFR (a); C2 = inclui pré-releases; . . .)	categoria	`dataset.csv`
RQ 08 (bônus)	Rework rate	releases corretivas ÷ releases classificadas pela heurística (sem exigir os 7 dias)	proporção	heurística

3.5. Inovações Propostas pelo Grupo (30% da nota)

Tabela 5. Inovações do grupo.

#	Inovação	Tipo	Por que é relevante	Onde aparece
I1	Decomposição do CFR (b) = oportunidade × condicional	(d) metodologia	Separa "publicar mais causa mais falhas" de "publicar mais cria mais pares release→release em 7 dias". Sem isso, a RQ 05 seria lida de forma errada.	Fig. 7, Seção 4.3, Conclusão
I2	Correlação parcial de Spearman contribuidores × idade	(d) controle de ameaça	Os dois fatores são correlacionados (ρ = 0,60); o parcial mostra qual deles carrega o efeito.	Fig. 11, Seção 4.3
I3	Inferência completa: Holm, IC 95%, ε^2 e δ de Cliff, contrastes planejados antes dos dados	(d) teste estatístico	Evita falso positivo por múltiplas comparações e reporta magnitude, não só p.	Tabelas 7 e 8
I4	Quatro versões da heurística de release corretiva avaliadas por F1	(b) variável	Permite escolher a heurística pela evidência da amostra-ouro, como pede o refinamento abaixo de F1 0,70.	Seções 4.1 e 4.3
I5	RQ 08 — rework rate	(a) nova RQ	Proporção de releases corretivas, sem a janela de 7 dias — mede retrabalho independente do ritmo de publicação.	Fig. 16, Tabela 12, Conclusão
I6	Pipeline reprodutível: cache atômico, retomada, parada antecipada, ordem runs→leadtime, 423 testes, conferência em Python	(c) arquitetura	Reduz dezenas de milhares de requisições e permite que outro grupo reproduza os números exatos.	Seções 3.3 e 4.1

4. Resultados

4.1. Coleta de Dados

Partimos dos 1.500 repositórios com mais estrelas (`stars:>1000`). Desses, 175 foram excluídos por não usarem GitHub Actions, sobrando 1.325. Outros 622 foram excluídos por terem menos de 5 releases publicadas na janela de observação, sobrando 703. Esses 703 foram avaliados em ordem decrescente de estrelas quanto ao critério de ao menos 50 workflow runs válidos no branch padrão. Durante essa avaliação, 46 foram excluídos por não atingirem o mínimo. Quando a meta de 300 repositórios elegíveis foi atingida, a avaliação parou, e os 357 restantes não chegaram a ser avaliados. A amostra final tem 300 repositórios (Figura 2).

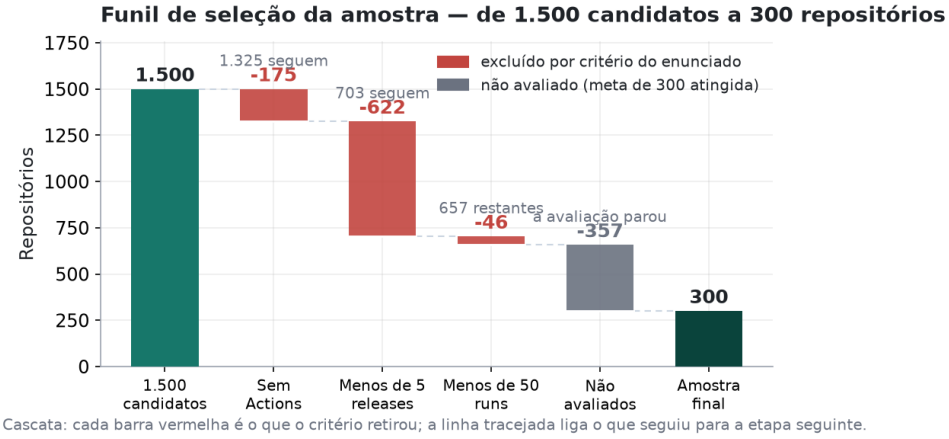


Figura 2. Funil de seleção da amostra. Cada barra vermelha é o que o critério retirou; a cinza, os 357 que não chegaram a ser avaliados porque a meta de 300 já havia sido atingida.

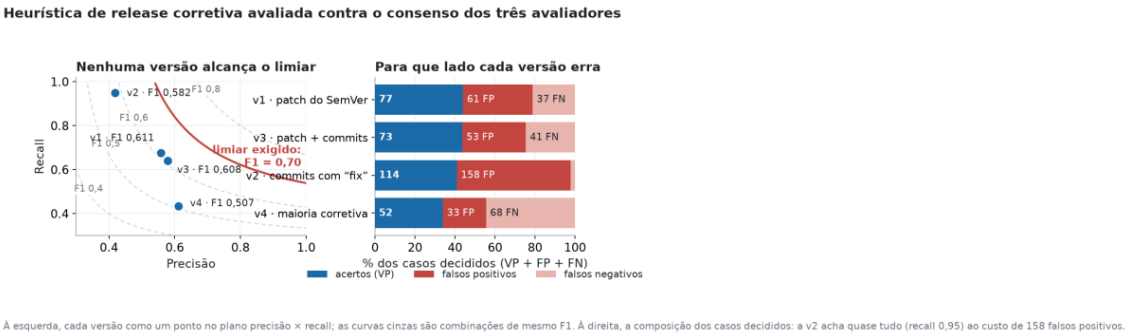
Nem toda métrica pôde ser medida em todos os 300: lead time (a) em **299** (um repositório sem release avaliável), CFR (b) em **298**, e tempo de recuperação em **283** — os outros **17** não tiveram nenhum episódio de falha com sucesso anterior, e por isso ficaram como "não medidos", e não como Elite ou Low. A proporção de episódios censurados por repositório tem mediana 0 e Q3 de 0,75%, ou seja, falhas nunca corrigidas dentro da janela são raras e não distorcem a RQ 04.

Validação manual. A amostra-ouro de 60 repositórios e 300 releases foi sorteada com semente 42, e o sorteio foi reproduzido com sucesso. Os três avaliadores concluíram a rotulagem de forma independente, cada um commitando apenas o próprio arquivo de rótulos. A concordância, medida pelo kappa de Fleiss [**?**] e interpretada segundo [**?**], foi quase perfeita para o tipo do projeto (κ = 0,952), substancial para release corretiva (κ = 0,771) e substancial para "releases representam entrega real" (κ = 0,611). Nesta última, a concordância observada é alta (96,7%) e o κ é puxado para baixo pela prevalência da categoria "sim" (95,6% dos rótulos): com uma categoria dominante, o acaso já explica boa parte do acordo. A dimensão que sustenta o CFR (b) — release corretiva — tem 88,9% de concordância observada, o que indica que o critério é claro o bastante para servir de gabarito.

Avaliadas contra o consenso dos três, **as quatro versões da heurística ficaram abaixo do limiar de 0,70** fixado a priori (Tabela 6 e Figura 3). O padrão dos erros é informativo: a v2 encontra quase toda release corretiva (recall 0,95), mas produz 158 falsos positivos — procurar "fix" nos commits captura quase qualquer release, porque quase toda release tem algum commit de correção. As versões que exigem mudança de patch no SemVer (v1 e v3) erram no sentido oposto, deixando passar um terço das corretivas. As duas pistas apontam para lados contrários, o que sugere que o refinamento não é ajustar um limiar, e sim combinar os sinais com peso. O caso a caso está em `data/processed/heuristica_erros.csv`.

Tabela 6. Heurística de release corretiva avaliada contra o consenso dos três avaliadores.

Versão da heurística	Precisão	Recall	F1	Releases avaliadas
v1 — patch do SemVer	0,558	0,675	**0,611**	270
v2 — commits com "fix"	0,419	0,950	0,582	300
v3 — patch + commits	0,579	0,640	0,608	270
v4 — maioria dos commits corretiva	0,612	0,433	0,507	300



À esquerda, cada versão como um ponto no plano precisão × recall; as curvas cinzas são combinações de mesmo F1. À direita, a composição dos casos decididos: a v2 acha quase tudo (recall 0,95) ao custo de 158 falsos positivos.

Figura 3. Heurística corretiva no plano precisão × recall, com curvas de mesmo F1 (à esquerda) e a composição dos casos decididos por versão (à direita). Nenhuma versão cruza a curva de F1 = 0,70.

4.2. Visualização Gráfica

Tabela 7. Estatística descritiva das métricas DORA (RQ 01 a RQ 04 e RQ 08).

RQ	Métrica	n	Mediana	IQR (Q1–Q3)	Mín – Máx
RQ 01	Deployment frequency (releases/semana)	300	**0,537**	0,249 – 1,227	0,096 – 28,17
RQ 02	Lead time (a), por release (dias)	299	**10,53**	3,02 – 39,21	0 – 2.109,7
RQ 02	Lead time (b), por commit (dias)	299	**7,76**	2,69 – 25,38	0 – 929,2
RQ 03	CFR (a), proxy de CI	300	**4,85%**	2,34% – 11,29%	0 – 93,3%
RQ 03	CFR (b), proxy de entrega	298	**18,75%**	2,42% – 40,0%	0 – 98,1%
RQ 04	Tempo de recuperação (horas)	283	**2,43**	0,95 – 6,28	0,07 – 181,8
RQ 08	Rework rate (releases corretivas)	292	**48,9%**	22,2% – 75,1%	—

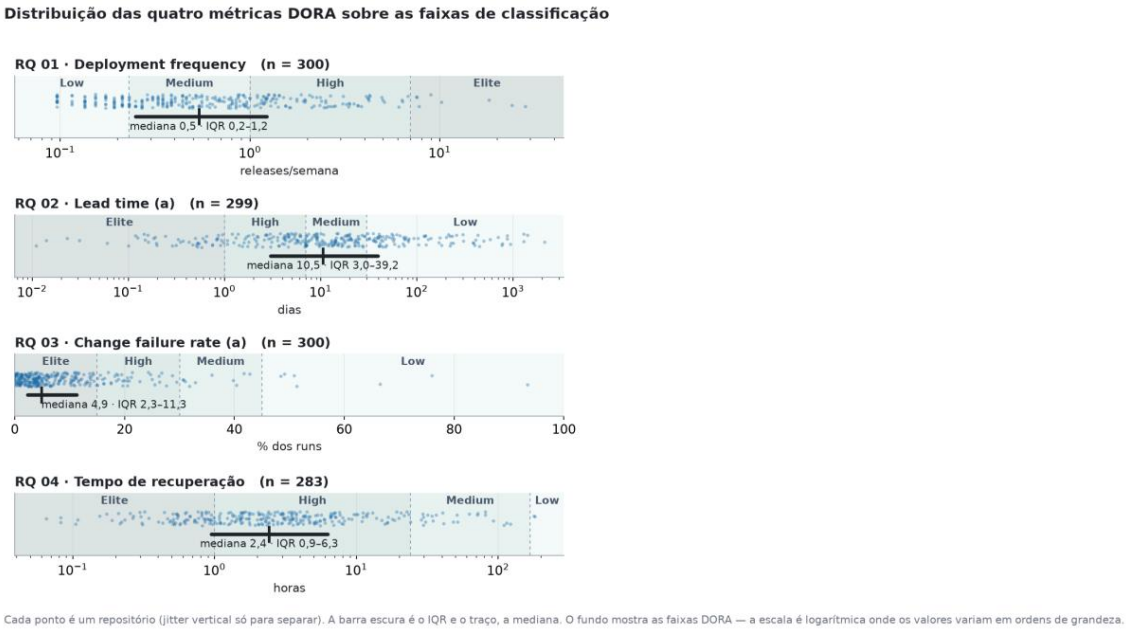


Figura 4. Distribuição das quatro métricas sobre as faixas DORA. Cada ponto é um repositório; a barra escura é o IQR e o traço, a mediana. A escala é logarítmica onde os valores variam em ordens de grandeza — é o que revela a assimetria que a Tabela 7 resume em dois números.

A Figura 4 mostra por que mediana e IQR são as estatísticas certas aqui: as quatro distribuições são assimétricas à direita e têm caudas longas — o lead time (a) vai de poucas horas a mais de 2.000 dias. Também deixa ver quanto cada repositório está longe do corte: no CFR (a) a massa se concentra bem à esquerda do limite de 15%, enquanto no lead time a mediana cai quase em cima do corte de 7 dias, o que explica a sensibilidade dessa métrica na RQ 07.

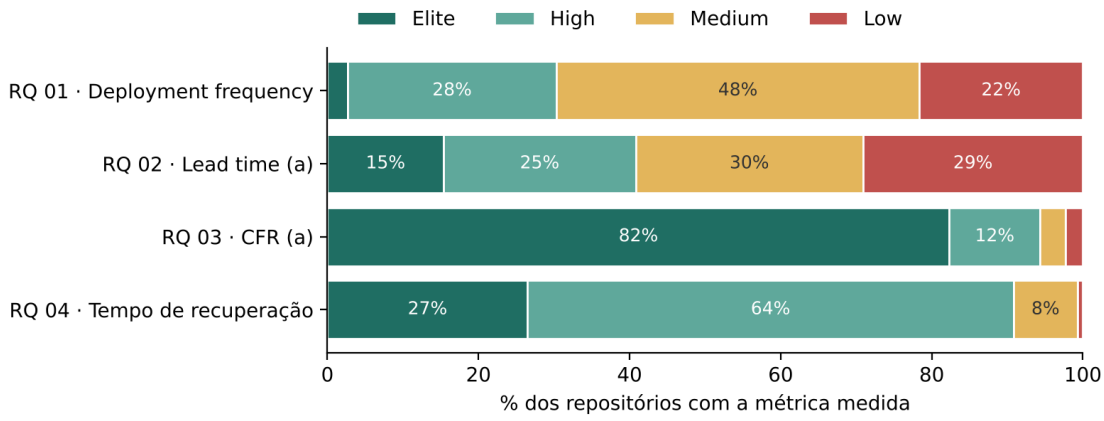


Figura 5. Distribuição das categorias DORA por métrica (combinação C1). Percentuais sobre os repositórios em que a métrica foi medida.

RQ 01 a RQ 04 — Como os repositórios se distribuem nas categorias DORA em cada métrica? Na frequência de deploy, quase metade (48,0%) é Medium e só 8 repositórios (2,7%) chegam a Elite. No lead time (a) a distribuição é a mais espalhada: 15,3% Elite e 29,0% Low. O CFR (a) concentra 82,3% em Elite ($\leq$ 15% de runs com falha), e a recuperação concentra 64,3% em High (entre 1 hora e 1 dia), sobre os 283 repositórios em que foi medida.

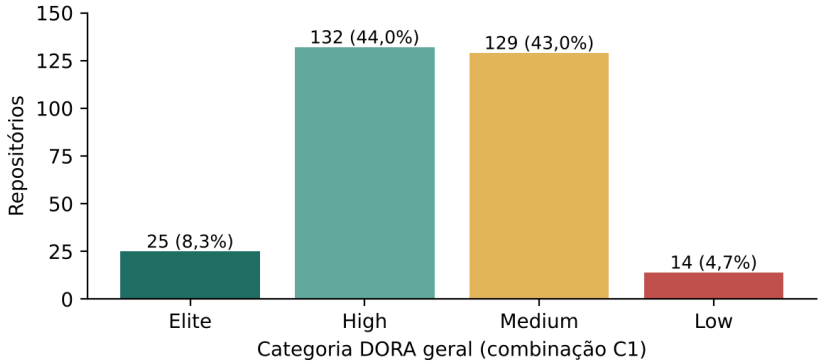


Figura 6. Categoria geral (mediana dos pontos das quatro métricas, arredondada para baixo).

Qual a categoria DORA geral dos repositórios? A amostra é essencialmente High (44,0%) ou Medium (43,0%); Elite (8,3%) e Low (4,7%) são extremos raros.

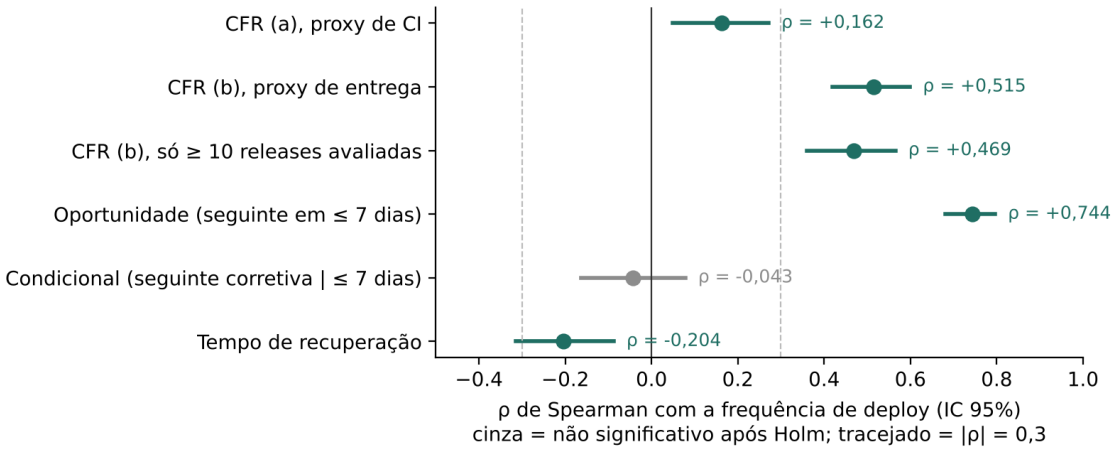


Figura 7. Correlação de Spearman entre a frequência de deploy e as métricas de estabilidade, com IC 95% e correção de Holm.

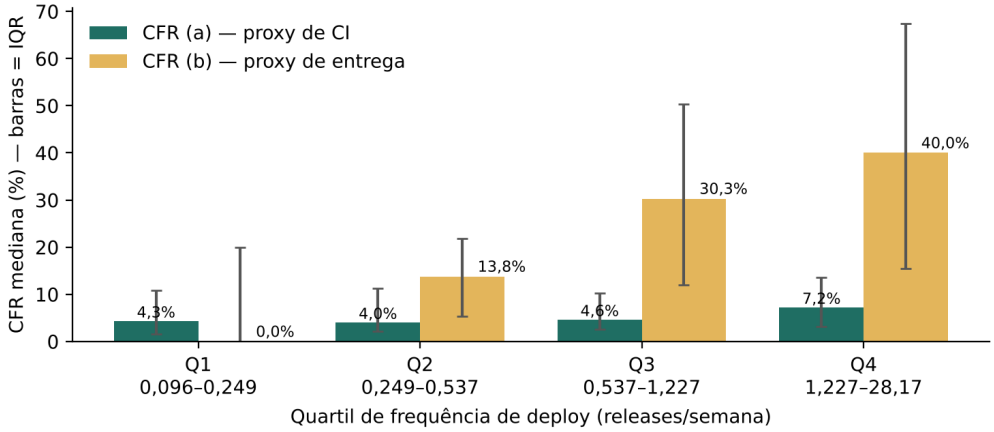


Figura 8. CFR (a) e (b) medianos por quartil de frequência de deploy. As barras de erro mostram o IQR.

RQ 05 — Repositórios com maior frequência de deploy apresentam maior ou menor taxa de falha? Com o CFR (a), a correlação é **fraca** (ρ = 0,162; IC 0,049 a 0,271; p Holm = 0,0096): a mediana sobe só de 4,3% no quartil menos frequente para 7,2% no mais frequente. Com o CFR (b) ela é **forte** (ρ = 0,515), e a mediana vai de 0% a 40%. A decomposição explica a diferença: a *oportunidade* (proporção de releases cuja seguinte sai em $\leq$ 7 dias) tem ρ = 0,744, enquanto a *condicional* (dessas, quantas a seguinte foi corretiva) tem $\rho = -0{,}043$, nula e não significativa. Por fim, publicar mais está associado a **recuperar mais rápido** ($\rho = -0{,}204$).

Tabela 8. Efeitos significativos na RQ 06 (Kruskal–Wallis e contrastes Mann–Whitney, Holm α = 0,05).

Fator	Métrica com efeito após Holm	Efeito	Leitura
Idade	Lead time (a)	ε^2 = 0,359 (grande); δ Q4×Q1 = 0,82	Q1 = 2,1 dias; Q4 = 42,7 dias
Idade	Deployment frequency	ε^2 = 0,106 (médio); $\delta = -0{,}49$	Q1 = 0,88/sem; Q4 = 0,36/sem
Idade	Pontuação DORA geral	ε^2 = 0,155 (médio); $\delta = -0{,}53$	Q1 mediana High; Q4 mediana Medium
Idade	Tempo de recuperação (contraste)	δ = 0,29 (pequeno); p Holm = 0,044	Q1 = 1,4 h; Q4 = 3,0 h
Contribuidores	Lead time (a)	ε^2 = 0,185 (médio); δ Q4×Q1 = 0,65	Q1 = 5,3 dias; Q4 = 40,9 dias
Contribuidores	Pontuação DORA (contraste)	$\delta = -0{,}31$ (pequeno)	Q1 High; Q4 Medium
Linguagem	Lead time (a)	ε^2 = 0,101 (médio)	Python 5,5 dias; C/C++ 73,9 dias
Estrelas	nenhuma	$\varepsilon^2 \leq 0{,}025$	Popularidade não discrimina desempenho

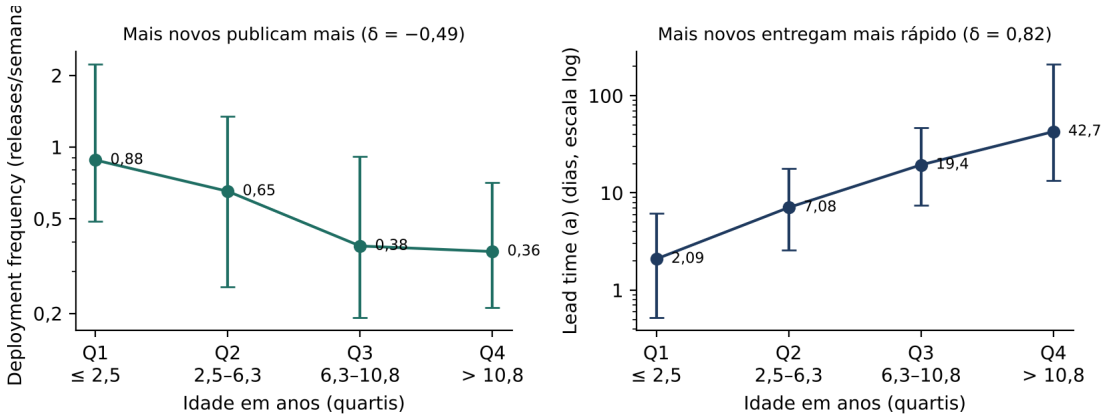


Figura 9. Frequência de deploy e lead time (a) por quartil de idade (mediana e IQR; escala logarítmica).

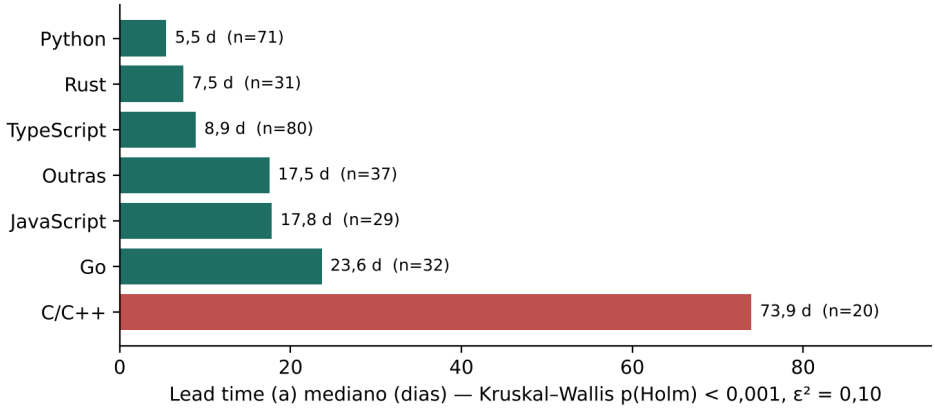


Figura 10. Lead time (a) mediano por linguagem principal, ordenado por valor.

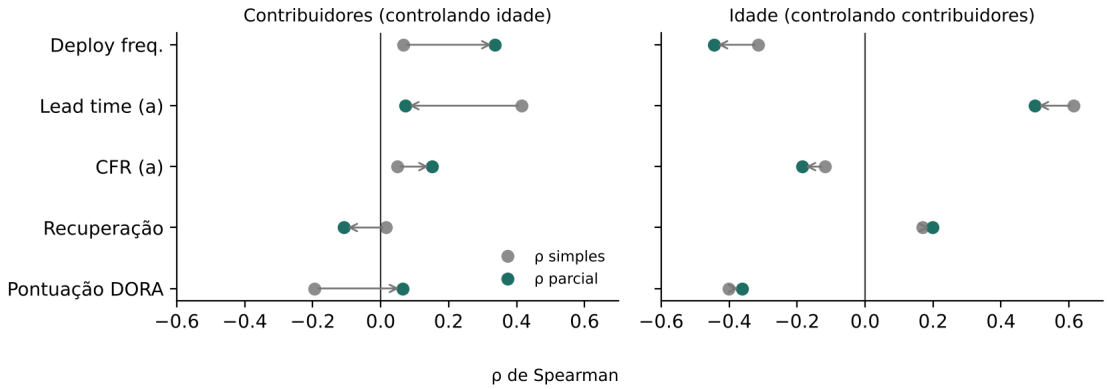


Figura 11. Spearman simples × parcial: o efeito de cada fator mantendo o outro constante (análise exploratória, fora das famílias de Holm).

RQ 06 — Quais características dos repositórios estão associadas a um melhor desempenho DORA? Contribuidores e idade são fortemente correlacionados ($\rho = 0{,}603$).

Controlando a idade, a associação de contribuidores com o lead time cai de $\rho = 0{,}415$ para 0,073 (p = 0,21) e a com a pontuação DORA some (0,065); por outro lado, aparece uma associação positiva com a frequência de deploy (0,066 $\rightarrow$ 0,337). Já a idade mantém seus efeitos depois de controlar contribuidores (lead time: 0,613 $\rightarrow$ 0,500; frequência: $-0{,}314$ $\rightarrow$ $-0{,}444$).

Tabela 9. Distribuição das categorias DORA em cada combinação de proxies (n = 300).

Combinação	Deploy	Lead time	CFR	Elite	High	Medium	Low
C1 (referência)	release	(a)	(a)	25	132	129	14
C2	release + pré-release	(b)	(b)	8	128	152	12
C3	tag	(b)	(a)	20	146	122	12

RQ 07 — A classificação DORA muda conforme a combinação de proxies? Entre um quinto e mais de um quarto dos repositórios muda de categoria conforme a combinação adotada, e nenhum par atinge a faixa "quase perfeita" de concordância (Tabela 10). As divergências são quase todas de uma única categoria (distância média $\approx$ 1), e não saltos entre extremos.

Tabela 10. Divergência entre as combinações (κ de Cohen ponderado, pesos lineares).

Par	Mudam de categoria	Distância média	κ ponderado	Concordância
C1 $\times$ C2	83 (27,7%)	1,08	0,566	moderada
C1 $\times$ C3	63 (21,0%)	1,02	0,703	substancial
C2 $\times$ C3	70 (23,3%)	1,00	0,653	substancial

Comparar combinações completas troca três definições ao mesmo tempo e não diz qual delas carrega a divergência. A Tabela 11 troca **uma de cada vez**, a partir de C1 (análise um-fator-por-vez), e separa quem sobe de quem desce de categoria.

Tabela 11. Contribuição de cada definição isoladamente, a partir de C1.

Troca aplicada a C1	Mudam	Sobem	Descem	κ ponderado
Change failure rate: (a) $\rightarrow$ (b)	62 (20,7%)	4	58	0,703
Lead time: (a) $\rightarrow$ (b)	45 (15,0%)	25	20	0,792
Unidade de deploy: release $\rightarrow$ tag	24 (8,0%)	15	9	0,886

O tempo de recuperação (RQ 04) não varia entre as combinações: ele vem dos workflow runs e não depende da unidade de deploy. Mantê-lo fixo isola o efeito do que de fato muda.

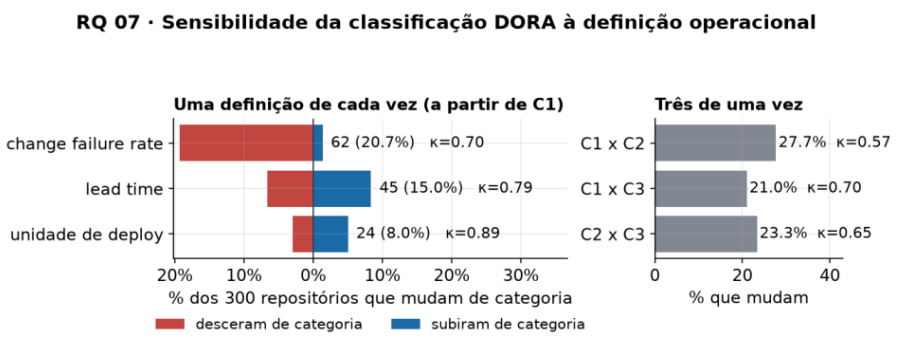


Figura 12. Sensibilidade da classificação à definição operacional. À esquerda, a análise um-fator-por-vez: cada linha troca uma única definição em C1, e as barras separam quem sobe de quem desce. À direita, as combinações completas. κ = kappa de Cohen ponderado.

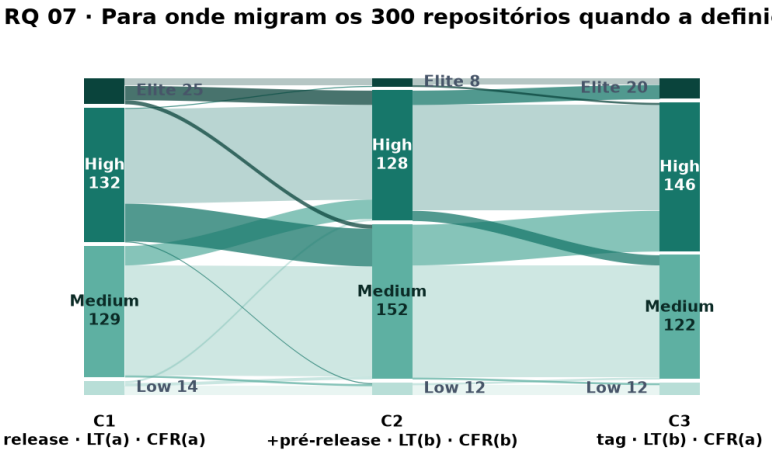


Figura 13. Migração dos 300 repositórios entre categorias nas três combinações (diagrama aluvial). Fitas claras permanecem na mesma categoria; fitas escuras mudam.

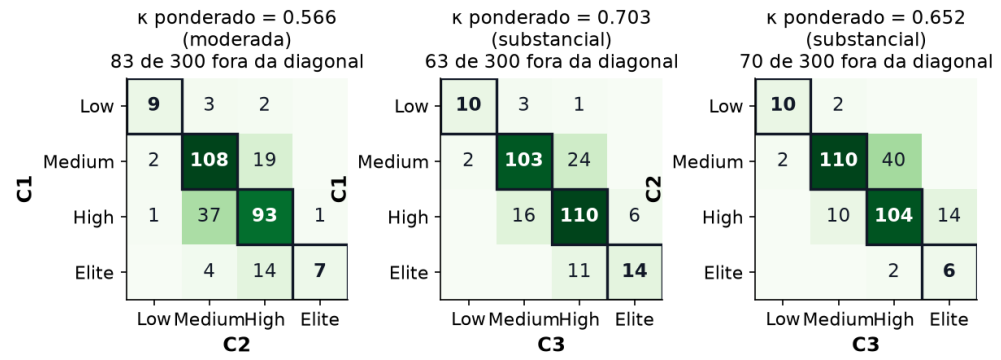


Figura 14. Matrizes de concordância entre as combinações. Os quadrados destacados são a concordância exata; como a escala é ordinal, o κ é ponderado — trocar Elite por High pesa menos que trocar Elite por Low.

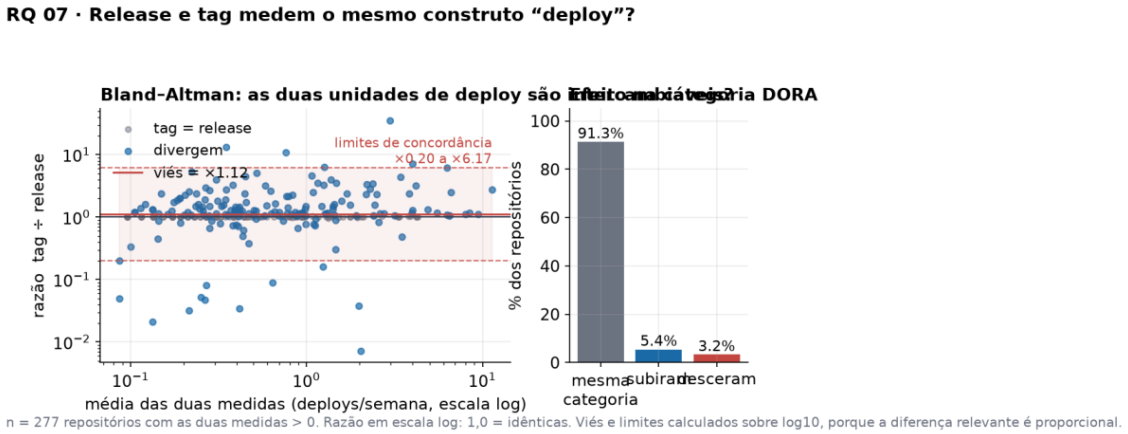


Figura 15. Bland–Altman entre as duas unidades de deploy. Viés de $\times$1,12 e limites de concordância de $\times$0,20 a $\times$6,17 (n = 277 repositórios com as duas medidas maiores que zero).

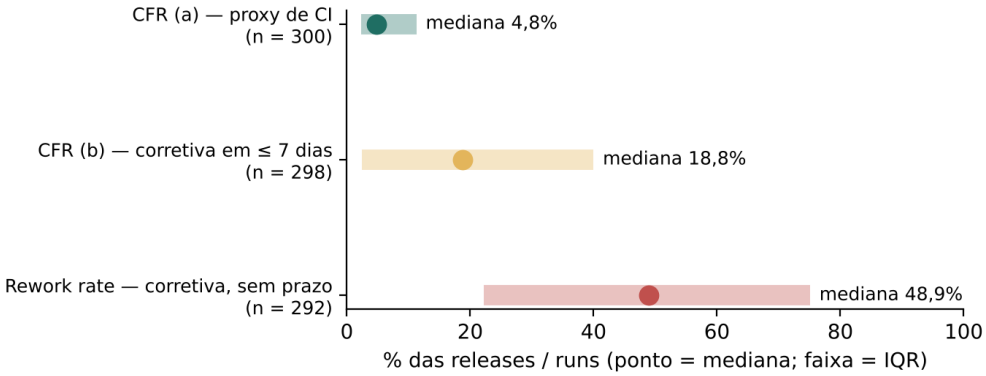


Figura 16. As três medidas de falha/retrabalho lado a lado: mediana e IQR por repositório.

Tabela 12. Correlações do rework rate (família de 4 pares, Holm α = 0,05).

Rework rate × …	ρ de Spearman	p (Holm)	Leitura
CFR (b), proxy de entrega	**0,52**	$<$ 0,0001	forte positiva
CFR (a), proxy de CI	$\lvert\rho\rvert \leq 0{,}04$	n.s.	nula
Deployment frequency	$\lvert\rho\rvert \leq 0{,}04$	n.s.	nula
Tempo de recuperação	$\lvert\rho\rvert \leq 0{,}04$	n.s.	nula

RQ 08 (bônus) — Qual a proporção de releases corretivas (rework rate)? A mediana do rework rate é **48,9%** (IQR 22,2% a 75,1%; 292 repositórios — os 8 sem nenhuma release classificada ficam fora). Só com $\geq$ 10 releases classificadas a mediana é 50,0% (240 repositórios), então o resultado não depende de repositórios com poucas releases. O rework se correlaciona fortemente com o CFR (b) (ρ = 0,52), mas **não** com a frequência de deploy, o CFR (a) ou o tempo de recuperação. Por quartil de frequência, a mediana fica estável entre 45% e 53% (Kruskal–Wallis p = 0,78). A diferença CFR (b) − rework tem mediana de −14,2 p.p. e é negativa em 255 de 291 repositórios.

4.3. Discussão

RQ 01 — H1 confirmada. A mediana de 0,54 release por semana (cerca de 2,3 por mês) coloca o repositório típico em Medium, e só 2,7% chegam a Elite. É o esperado para projetos que versionam por release: publicar uma versão exige changelog, tag e, muitas vezes, publicação em registro de pacotes, e não acontece a cada merge como num deploy contínuo de produção. Assim, a frequência de releases subestima a frequência real de integração desses times.

RQ 02 — H2 confirmada. O lead time (a) tem mediana de 10,5 dias (Medium) e o (b) de 7,8 dias, menor como previsto: na variante (a), um único commit antigo que "esperou" várias releases define o valor da release inteira; na (b) ele pesa como um commit entre muitos. O máximo de 2.109 dias na (a) contra 929 na (b) mostra o mesmo efeito nos extremos. Commits com data posterior à release (rebase/squash) foram contados em `negative_lead_time_commits` e são uma ameaça de construto.

RQ 03 — H3 confirmada. As variantes divergem fortemente: mediana de 4,9% no CFR (a) contra 18,8% no CFR (b). Pelo CFR (a), 82,3% dos repositórios seriam Elite — mas ele mede falha de pipeline, e um projeto pode ter CI estável e ainda assim precisar de releases corretivas logo após publicar. O CFR (b) se aproxima mais do conceito do DORA (mudança que chegou ao usuário e precisou de correção), porém depende da heurística de release corretiva, validada contra a amostra-ouro com F1 de apenas 0,611 na melhor versão. A conclusão sobre estabilidade, portanto, depende muito do proxy escolhido — exatamente o que a RQ 07 deve quantificar.

RQ 04 — H4 confirmada. A recuperação mediana é de 2,4 horas, e 90,8% dos repositórios medidos são Elite ou High. Pipelines quebrados no branch padrão são corrigidos em horas. A censura é baixa (mediana 0), então descartá-la não infla o resultado. Fica a ressalva de construto: "recuperar o CI" não é o mesmo que restaurar um serviço em produção.

RQ 05 — H5 confirmada. Não há trade-off relevante entre velocidade e estabilidade. Com o CFR (a), a correlação é positiva mas fraca (ρ = 0,16, abaixo do corte de 0,3 da hipótese). Com o CFR (b), o ρ = 0,52 parece indicar que quem publica mais falha mais, mas a decomposição mostra que ele vem inteiro da oportunidade (ρ = 0,74): quem publica mais tem, por construção, mais releases seguidas de outra em até 7 dias. Dado que a seguinte saiu em até 7 dias, a chance de ela ser corretiva não muda com a frequência ($\rho = -0{,}04$, não significativo). Além disso, quem publica mais recupera o CI mais rápido ($\rho = -0{,}20$). O quadro é coerente com o argumento do Accelerate [**?**] de que velocidade e estabilidade andam juntas. Em linguagem simples: o CFR (b) "pune" quem publica com frequência por um efeito aritmético da própria definição, e um grupo que não fizesse a decomposição concluiria o contrário.

RQ 06 — H6 parcialmente confirmada. O fator mais forte é a idade, e não a linguagem ou a popularidade. Projetos mais jovens publicam mais (0,88 $\times$ 0,36 releases/semana; parte confirmada da H6) e têm lead time vinte vezes menor (2,1 $\times$ 42,7 dias, efeito grande). A previsão sobre contribuidores foi contrariada no teste direto — o quartil com mais contribuidores tem lead time maior (40,9 $\times$ 5,3 dias) —, mas a correlação parcial mostra que isso é quase todo efeito da idade: projetos antigos acumulam contribuidores. Mantida a idade constante, mais contribuidores se associam a mais deploys (ρ parcial = 0,34), na direção prevista pela H6, e também a um CFR (a) um pouco maior (0,15). Na linguagem, só o lead time difere após Holm (Python 5,5 dias, C/C++ 73,9 dias); TypeScript publica três vezes mais que C/C++ (δ = 0,35, efeito médio [**?**]), mas com 20 repositórios C/C++ o teste não tem poder para confirmar a diferença após Holm (p = 0,21). As estrelas não se associam a nenhuma métrica.

RQ 07 — H7 confirmada, inclusive na parte específica. A hipótese previa que a categoria mudaria para pelo menos 20% da amostra em alguma combinação e que o CFR seria a métrica mais sensível. As duas previsões se confirmam: a divergência vai de 21,0% (C1 $\times$ C3) a 27,7% (C1 $\times$ C2), e nenhum par atinge concordância quase perfeita. A análise

um-fator-por-vez (Figura 12) mostra qual escolha carrega essa divergência. Trocar apenas o CFR de (a) para (b) muda a categoria de 62 repositórios (20,7%), e em 58 deles para baixo — o efeito previsto pela H7: o CI quebra pouco (mediana de 4,9%), mas releases corretivas são frequentes (18,8%), e a faixa Elite exige $\leq 15\%$. Trocar apenas o lead time muda 45 (15,0%), em direções equilibradas, e trocar apenas a unidade de deploy muda 24 (8,0%) — o menor efeito dos três.

Esse último resultado merece atenção: release e tag são quase intercambiáveis nesta amostra. O Bland–Altman (Figura 15) mostra viés de apenas $\times 1{,}12$ entre as duas medidas de frequência de deploy, ou seja, contar tags dá em média 12% mais deploys que contar releases. Os limites de concordância, porém, são largos ($\times 0{,}20$ a $\times 6{,}17$): há repositórios em que as duas medidas diferem por um fator de cinco, nos dois sentidos. As duas unidades concordam no agregado, mas não são substituíveis repositório a repositório — distinção que só aparece quando se mede o acordo, e não apenas a tendência central.

A parte da H7 sobre pré-releases foi contrariada: incluí-las em C2 não elevou a classificação. C2 é a combinação mais severa (apenas 8 Elite, contra 25 em C1), porque o ganho de frequência de deploy é anulado pela troca simultânea do CFR para a variante (b) — o diagrama de migração (Figura 13) mostra os repositórios Elite de C1 escorrendo para High em C2. Isso ilustra por que a análise um-fator-por-vez é necessária: comparando só as combinações completas, o efeito do CFR e o das pré-releases se confundem. Como o CFR é o eixo mais sensível e a heurística que o sustenta não atingiu F1 de 0,70, a classificação em C2 deve ser lida com reserva; a conclusão robusta é a de C1 e C3, que compartilham o CFR (a) — a amostra é majoritariamente High/Medium, com Elite e Low raros.

RQ 08 (bônus, exploratória). Cerca de metade das releases de um repositório típico é classificada como corretiva — bem acima dos 18,8% do CFR (b). A diferença vem da janela de 7 dias: em 255 de 291 repositórios o CFR (b) é menor que o rework, ou seja, boa parte das correções sai mais de uma semana depois da release anterior e por isso não conta como falha no CFR (b). O resultado mais importante é que o retrabalho não cresce com a frequência de deploy ($\rho \approx 0$; quartis estáveis entre 45% e 53%). Isso confirma por outro caminho a conclusão da RQ 05: a correlação entre CFR (b) e frequência vem da janela de 7 dias da própria métrica, e não de mais defeitos em quem publica mais. A ressalva é a mesma do CFR (b): os números dependem da heurística v3, cujo F1 contra a amostra-ouro ficou em 0,608 — um rework de $\sim$50% pode indicar que a heurística marca como corretivas releases de manutenção com algum commit "fix", o que a precisão de 0,579 da v3 corrobora.

Ameaças à validade. *Construto*: release $\neq$ deploy em produção; falha de CI $\neq$ falha em produção; datas de commit podem ser reescritas por rebase; e o filtro `event = push` não cobre todo o CI — `ollama/ollama` e `microsoft/markitdown` retornam zero runs com `branch=main&event=push`, mas 119 e 354 runs sem o filtro de evento, porque rodam CI por `pull_request` e por eventos dinâmicos, de modo que o CFR (a) e o tempo de recuperação medem apenas a fatia do CI disparada por push. A heurística corretiva foi validada contra a amostra-ouro, mas a melhor versão alcançou F1 de apenas 0,611, abaixo do limiar de 0,70, o que mantém o CFR (b) e a RQ 08 como resultados

provisórios. *Interna*: idade e contribuidores são confundidos (tratado pela correlação parcial, mas outros confundidores ficaram de fora); além disso, 45 dos 300 repositórios atingiram o teto de 150 datas de commit na coleta de tags, então a frequência de deploy da combinação C3 neles é um piso, e não o valor exato. *Externa*: a amostra é de 300 repositórios muito populares com GitHub Actions e releases frequentes; o resultado não se generaliza para projetos pequenos ou empresariais. *Conclusão*: com 20 repositórios C/C++, os contrastes de linguagem têm baixo poder [**?**]; a correção de Holm deixa os resultados conservadores.

O que as inovações acrescentaram. A decomposição do CFR (I1) inverteu a leitura ingênua da RQ 05: sem ela, o resultado mais forte do enunciado ($\rho = 0{,}52$) seria interpretado como trade-off. O rework rate (I5) confirmou essa leitura por uma métrica independente da janela de 7 dias: o retrabalho não varia com a frequência de deploy. A correlação parcial (I2) mostrou que a contrariedade da H6 sobre contribuidores é um efeito de confusão com a idade. Holm e tamanhos de efeito (I3) reduziram de 10 para 5 os testes omnibus significativos (de 20), evitando apresentar como achado o que seria ruído de múltiplas comparações.

5. Conclusão

Medidos por rastros do GitHub, os 300 repositórios populares estudados são, em sua maioria, High ou Medium na escala DORA: publicam cerca de duas releases por mês, levam de uma semana a um mês para levar uma mudança até uma release e recuperam o CI em poucas horas. A taxa de falha é a métrica mais sensível ao proxy: pelo CI quase todos são Elite, pela sequência de releases corretivas a mediana fica perto de 19%.

Publicar com mais frequência não está associado a falhar mais. A correlação aparente com o CFR (b) é produzida pela própria definição da métrica, e desaparece quando se olha a probabilidade de a release seguinte ser corretiva; quem publica mais ainda se recupera mais rápido. O rework rate (RQ 08) reforça isso: cerca de metade das releases é corretiva, mas essa proporção é a mesma em repositórios que publicam pouco ou muito. Entre as características do repositório, a idade é a que mais pesa — projetos jovens são mais rápidos —, enquanto o número de contribuidores só parece prejudicar porque acompanha a idade, e a popularidade não importa.

Limitações: a heurística de release corretiva foi validada contra a amostra-ouro, mas nenhuma das quatro versões atingiu o limiar de F1 de 0,70 (a melhor, a v1, ficou em 0,611), de modo que o CFR (b) e o rework rate (RQ 08) permanecem provisórios até o refinamento; a análise de sensibilidade mostra que a classificação depende da combinação de proxies, com até 27,7% dos repositórios mudando de categoria; e a amostra é de projetos muito populares, em uma única janela de 12 meses.

Com mais tempo, o grupo concluiria a validação e escolheria a heurística pelo maior F1, mediria deploys reais (publicações em npm/PyPI/crates.io e eventos `deployment` da API) em vez de releases, e expandiria as inovações I1 e I2: a decomposição oportunidade $\times$ condicional deveria ser o padrão para qualquer CFR baseado em janela de tempo, e um modelo multivariado (idade, contribuidores, linguagem e tipo do projeto juntos) substituiria as correlações parciais par a par.

Referências

Boulic, R. and Renault, O. (1991). 3d hierarchies for animation. In Magnenat-Thalmann, N. and Thalmann, D., editors, *New Trends in Animation and Visualization*. John Wiley & Sons ltd.

Knuth, D. E. (1984). *The TEX Book*. Addison-Wesley, 15th edition.

Smith, A. and Jones, B. (1999). On the complexity of computing. In Smith-Jones, A. B., editor, *Advances in Computer Science*, pages 555–566. Publishing Press.